Functions

Lecture 4 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 Why functions? 3

2 Defining and calling functions 4
2.1 Syntax . . . 4
2.2 A first example: cat years and human years . . . 5
2.3 A second example: the factorial . . . 6
2.4 Functions are objects . . . 7

3 Returning values 8
3.1 `print` versus `return` . . . 8
3.2 Every function returns something . . . 8
3.3 Returning several values . . . 9

4 Passing arguments to functions 10
4.1 Positional arguments . . . 10
4.2 Keyword arguments . . . 11
4.3 Default argument values . . . 12
4.4 Special parameters: positional-only and keyword-only . . . 13

5 Scope and how arguments are passed 16
5.1 Local variables and scope . . . 16
5.2 What happens to an argument? . . . 17

6 Functions with any number of arguments 20
6.1 Extra positional arguments: `*args` . . . 20
6.2 Unpacking argument lists with `*` . . . 22
6.3 Extra keyword arguments: `**kwargs` . . . 22
6.4 Unpacking dictionaries with `**` . . . 24

Summary 24

Exercises 25
Exercise 1: A guessing game . . . 25
Exercise 2: Projectile motion as a function . . . 25

Exercise 3: Resistors in series and in parallel 25
Exercise 4: Functions with and without side effects 26

Further reading **26**

Learning objectives

By the end of this lecture, you should be able to:

- explain why functions are essential for writing larger programs;
- define functions with `def`, document them with docstrings, and call them;
- return one or several values from a function, and explain the difference between `print` and `return`;
- pass arguments by position and by keyword, and give parameters default values;
- explain the scope of variables and what happens to an argument that is passed into a function;
- write functions that accept any number of arguments with `*args` and `**kwargs`.

1 Why functions?

Stand on the shoulders of giants. We have been using functions all along, and with such ease that we may not have noticed how important and widespread they are. For example, `print()` and `len()` are built-in functions, while `append()` and `copy()` are functions attached to lists (we call such functions *methods*). Without functions, we would have to write every program from scratch, and we could never build on the work of the experts who came before us.

Break a big problem down into smaller tasks. Solving a complicated problem with separate functions is comparable to how a company works towards its goals. The CEO shares a vision with the heads of department, who make a plan and divide the work among their employees according to each person's strengths. Each employee carries out their own task without needing to know exactly how their colleagues do theirs, and the head of department supervises the outcomes without needing to understand every detail — otherwise, they might as well do all the work themselves. Together, the team achieves its goal.

Writing complex software, or solving a fairly complex scientific problem, works in the same way. The overall problem is broken down into sub-problems, which may be broken down further into sub-sub-problems, and so on. Each person focuses on their own sub-problems and wraps the solutions up as **functions**, which other people can use easily and effectively without having to understand the details. A function receives a set of inputs, carries out a specific task and returns the result. In this way, we can exploit each other's expertise, and the workflow stays smooth.

Reuse code. Even someone who can solve a big problem alone needs a strategy for reusing the code they have already written. Imagine that we have many arrays of numbers that must be sorted in ascending order. With two or three arrays, we could copy and paste the sorting code for each of them. With a few hundred arrays, however, this would quickly become extremely troublesome — and it would be impossible if we did not know in advance how many arrays there will be. Copied code is also hard to maintain: if you find a bug, you have to fix it in every copy.

A first taste. Recall the rectangle that we drew in Lecture 2. Let us give the job of drawing one line a name, by defining a function `draw_line()`:

```
def draw_line(end_char, width):
    """Print one line of the rectangle: end_char, (width - 2) dashes,
     ↪  end_char."""
    print(end_char + "-" * (width - 2) + end_char)

h, w = 5, 12
for j in range(1, h + 1):
    if j == 1 or j == h:
        draw_line("x", w)
    else:
        draw_line("|", w)
```

```
x----------x
|----------|
|----------|
|----------|
x----------x
```

The main loop now reads almost like the pseudo-code, and `draw_line()` can be reused anywhere, e.g. `draw_line("+", 20)`.

2 Defining and calling functions

2.1 Syntax

```
# note the keyword def and the colon ":"
def function_name(parameter_1, parameter_2, ...):
    """One-line summary of what the function does.

    A more detailed explanation can follow after a blank line, and it can
 ↪  be
    as long as you like: describe the inputs, the output, the algorithm,
 ↪  the
    data structures it uses, and so on.
    """
    <statement-1>
    ...
    <statement-N>
```

```
    return result
```

- The keyword `def` introduces a **function definition**. It must be followed by the function name and the parenthesised list of **parameters**.
- The statements that form the **body** of the function start on the next line, after the colon `:`, and must be indented.
- The first statement of the body can *optionally* be a string literal, called the **docstring**, which documents the function. Docstrings are usually written with triple double quotes, `"""..."""`, so that they can span several lines. Whatever you write there is displayed by `help(function_name)` (or, in Jupyter, by `function_name?`).
- The `return` statement ends the function and hands the value of `result` back to the caller as the **output** of the function. If a function has no `return` statement, it returns the special value `None`.
- A function definition binds the function name to a **function object**. Other names can refer to that *same* function object and can also be used to call the function.



Parameters and arguments

The names in the function definition, such as `parameter_1`, are called **parameters**. The actual values passed when the function is called are called **arguments**. In everyday speech, the two words are often used interchangeably.

2.2 A first example: cat years and human years

We define a function that converts the age of a cat into the equivalent age of a human. We display its docstring with `help()` and then call the function.

```
def cat_age_to_human_age(cat_age):
    """Convert the age of a cat into the (approximately) equivalent human
     ↪  age.

    A one-year-old cat corresponds to a 16-year-old human and a
 ↪  two-year-old
    cat to a 24-year-old human; after that, every cat year adds about four
    human years. This is only a rough rule of thumb!
    """
    multiplier = 4
    if cat_age == 1:
        return 16
    if cat_age == 2:
        return 24
    return multiplier * cat_age + 16
```

```
help(cat_age_to_human_age)
```

```
Help on function cat_age_to_human_age in module __main__:

cat_age_to_human_age(cat_age)
    Convert the age of a cat into the (approximately) equivalent human age.

    A one-year-old cat corresponds to a 16-year-old human and a two-year-old
    cat to a 24-year-old human; after that, every cat year adds about four
    human years. This is only a rough rule of thumb!
```

```
# the result is neither stored nor shown, so this line is useless here
cat_age_to_human_age(12)
felix_age = cat_age_to_human_age(4)  # store the returned value in a
 ↪ variable ...
print(felix_age)                     # ... and then print it

print(cat_age_to_human_age(12))  # of course, you can also print the result
 ↪ directly
```

```
32
64
```

> **Jupyter shortcuts**
>
> In Jupyter, typing `cat_age_to_human_age?` in a cell, or pressing **Shift + Tab** after the function name, also shows the docstring. This works for every function, e.g. `print?`.

2.3 A second example: the factorial

The factorial of a non-negative integer n is defined by $n! = 1 \times 2 \times \cdots \times n$, with $0! = 1$.

```
def factorial(n):
    """Return the factorial n! = 1 x 2 x ... x n of a non-negative integer
     ↪ n."""
    if n < 0:
        print("The factorial is only defined for non-negative integers!")
    else:
        result = 1
        for i in range(1, n + 1):
```

```
            result = result * i
        return result
```

```
# For n < 0, the function reaches no return statement, so it returns None.
print(factorial(-1))
print("--------------------------")

for i in range(0, 5):
    print("{0}! = {1}".format(i, factorial(i)))
```

```
The factorial is only defined for non-negative integers!
None
--------------------------
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
```

2.4 Functions are objects

As explained above, a function definition binds the function name to a function object. We can bind another name to the same object: for example, we can assign the function `factorial` to a new variable `f` and then use `f` in exactly the same way. In Lecture 5, we will see that a function is indeed an object — a *function object* — so `factorial` and `f` are simply two names, or two aliases, for the same object.

```
f = factorial      # no brackets: we refer to the function itself, we do
 ↪  not call it
print("0! =", f(0))
print("4! =", f(4))
print(f is factorial)
```

```
0! = 1
4! = 24
True
```

3 Returning values

3.1 `print` versus `return`

Beginners often confuse **printing** a value with **returning** it. `print()` merely *displays* a value in the output area; the caller cannot use it in further calculations. `return` *hands the value back* to the caller, who can store it, print it or compute with it.

```
def print_larger(a, b):
    """Print the larger of a and b (and return nothing)."""
    if a > b:
        print(a)
    else:
        print(b)

def larger(a, b):
    """Return the larger of a and b."""
    if a > b:
        return a
    else:
        return b

result_1 = print_larger(3, 7)  # displays 7, but returns None
result_2 = larger(3, 7)        # displays nothing, but returns 7
print(result_1, result_2)
# a returned value can be used in further calculations
print(larger(3, 7) * 10)
```

```
7
None 7
70
```

> **!** Prefer `return`
>
> A function that *returns* its result can be reused in many situations; a function that only *prints* its result cannot. Unless the purpose of a function is to display something, let it return the result and leave the printing to the caller.

3.2 Every function returns something

- Every function, just like a function in mathematics, returns exactly one thing, by means of the `return` statement.

- Even if we do not return anything explicitly, a Python function actually returns the value `None`.

```
def addition(a, b):
    """Is there a duller function?"""
    return a + b

s = addition(3, 7)     # its use is dull too
print(s)

def say_hello(name):
    print("Hello", name, "!")

output = say_hello("Khiem")
print(output)          # say_hello() has no return statement, so it returns
 ↪  None
```

```
10
Hello Khiem !
None
```

3.3 Returning several values

To return several outputs from one function, we return a tuple, a list or a dictionary; the purpose of the function usually suggests which one is best. Returning a **tuple** is the most common choice, because the caller can unpack it immediately (Lecture 3).

```
def find_min_max(array):
    """Return the smallest and the largest element of a non-empty list."""
    min_val, max_val = array[0], array[0]
    for i in range(1, len(array)):
        if array[i] < min_val:
            min_val = array[i]
        if array[i] > max_val:
            max_val = array[i]
    return min_val, max_val          # packs the two values into a tuple

min_val, max_val = find_min_max([-1, -4, 3, 5, 2])    # unpacks the tuple
print("min_val =", min_val)
print("max_val =", max_val)
print(find_min_max(list(range(0, 20, 2))))
```

```
min_val = -4
```

```
max_val = 5
(0, 18)
```

4 Passing arguments to functions

We now study the basics of passing input arguments to a function:

1. **positional arguments**, where the *position* of each argument in the call decides which parameter it is assigned to;
2. **keyword arguments**, where we state explicitly which parameter receives which value, regardless of the order;
3. **default argument values**, which allow us to call a function with fewer arguments than it is defined to accept;
4. **special parameters**, which restrict how arguments may be passed: positional-only and keyword-only.

4.1 Positional arguments

By default, the arguments of a function call are assigned to the parameters in order: the first argument to the first parameter, and so on. The positions of the arguments are therefore essential.

In the following example, we define a function that solves the equation $ax + b = 0$ for x, where a and b are two real numbers. The solution is $x = -b/a$, provided that $a \neq 0$. The function receives two arguments, first a and then b, and clearly their order matters.

```
def solve_linear_equation(a, b):
    """Solve the equation a*x + b = 0 for x."""
    if a != 0:
        return -b / a
    else:
        if b == 0:
            print("The equation has infinitely many solutions.")
        else:
            print("The equation has no solution.")
```

```
print("Solve the equation 2 x + 1 = 0:", solve_linear_equation(2, 1))
print("Solve the equation   x + 2 = 0:", solve_linear_equation(1, 2))
print("Solve the equation   x     = 0:", solve_linear_equation(1, 0))
print("--------------------------")
print("Solve the equation 0 x + 1 = 0:", solve_linear_equation(0, 1))
```

```
Solve the equation 2 x + 1 = 0: -0.5
Solve the equation   x + 2 = 0: -2.0
Solve the equation   x     = 0: 0.0
-------------------------
The equation has no solution.
Solve the equation 0 x + 1 = 0: None
```

Note the order of the output in the last case: Python evaluates the arguments of `print()` — including the call to `solve_linear_equation()`, which prints its message — before `print()` itself displays anything. The function then returns `None`.

4.2 Keyword arguments

When we know the names of the parameters, we can state which parameter receives which value, using **keyword arguments** of the form

```
kwarg=value     # kw stands for keyword, arg for argument
```

Returning to the equation $ax + b = 0$, we only need to write `a=...` and `b=...` in the function call. The order then no longer matters. Of course, the values can also be given by variables.

```
print("Solve the equation 2 x + 1 = 0:", solve_linear_equation(b=1, a=2))
print("Solve the equation 2 x + 1 = 0:", solve_linear_equation(2, 1))
# These two calls are equivalent, because a and b receive the same values.

a, b = 2, 1
# In "a=a" and "b=b", the names on the left of "=" are the PARAMETERS of
 ↪  the
# function, while the names on the right are our VARIABLES, which hold the
 ↪  values.
print("Solve the equation 2 x + 1 = 0:", solve_linear_equation(b=b, a=a))

# Of course, the variables do not have to be called a and b.
c1, c2 = 2, 1
print("Solve the equation 2 x + 1 = 0:", solve_linear_equation(a=c1, b=c2))
```

```
Solve the equation 2 x + 1 = 0: -0.5
Solve the equation 2 x + 1 = 0: -0.5
Solve the equation 2 x + 1 = 0: -0.5
Solve the equation 2 x + 1 = 0: -0.5
```

This example does not really show the benefit of keyword arguments, because there are only two parameters, and the names `a` and `b` carry no clear meaning. Keyword arguments become extremely useful for functions with a long list of parameters whose names *do* carry meaning, such as `print(...,`

`sep=" | ", end="")`. Most modern IDEs also help you to type keyword arguments with their auto-complete feature (**Tab**).

4.3 Default argument values

The most useful way to write a function that can be called with a variable number of arguments is to give one or more parameters a **default value**, with the syntax `parameter=value` in the definition. The function can then be called with **fewer arguments** than it is defined to accept: any parameter that does not receive a value in the call takes its default value, without raising an error.

In the following example, we define a function with three parameters; the second and the third have default values.

```
def ask_ok(prompt, retries=4, reminder='Come on! Only "yes" or "no".'):
    """Keep asking a question until the user answers 'yes' or 'no'.

    Return True for yes and False for no. After too many invalid answers,
    print the reminder and return False.
    """
    while True:
        ok = input(prompt)
        if ok in ('y', 'ye', 'yes'):
            return True
        if ok in ('n', 'no', 'nah'):
            return False
        retries = retries - 1
        if retries < 0:
            print(reminder)
            return False
```

This function can be called in several ways:

```
ask_ok("Do you want a quiz? ")             # only the mandatory argument
ask_ok("Do you want a quiz? ", retries=2)  # override one default value
# all three, by position
ask_ok("Do you want a quiz? ", 2, "Please type yes or no!")
# all by keyword
ask_ok(prompt="Yes or no? ", retries=1, reminder="Please type only 'yes' or
 ↪  'no'.")
```

⚠ Never use a mutable default value

A default value is created **once**, when the function is defined, and not every time the function is called. If the default is a mutable object such as a list, every call that relies on the default shares the *same* list, which leads to surprising results. Use `None` as the default instead, and create a new list inside the function. (The section on how arguments are passed explains why mutability matters.)

```
def add_item_bad(item, basket=[]):  # DON'T: the same list is reused in
 ↪  every call!
    basket.append(item)
    return basket

print(add_item_bad("apple"))
print(add_item_bad("banana"))          # surprise: the apple is still there

def add_item(item, basket=None):       # DO: use None and create a new list
 ↪  inside
    if basket is None:
        basket = []
    basket.append(item)
    return basket

print(add_item("apple"))
print(add_item("banana"))
```

```
['apple']
['apple', 'banana']
['apple']
['banana']
```

4.4 Special parameters: positional-only and keyword-only

By default, arguments may be passed to a Python function either by position or explicitly by keyword: by default, all parameters are **positional-or-keyword** parameters. For readability, it can make sense to restrict the way in which arguments can be passed, so that a developer only needs to look at the function definition to know whether items are passed by position, by position or keyword, or by keyword only. A function definition may look like this:

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
      ----------     ----------     ----------
```

```
          |               |              |
          |      positional or keyword   |
          |                              +-- keyword only
          +-- positional only
```

where / and * are optional. If they are used, these symbols indicate how arguments may be passed to the parameters:

- **Positional-or-keyword parameters.** If neither / nor * is present, arguments may be passed by position or by keyword. This is the default.
- **Positional-only parameters** are placed *before* a forward slash /. Their order matters, and they **cannot** be passed by keyword.
- **Keyword-only parameters** are placed *after* an asterisk *. They **must** be passed by keyword.

Positional-or-keyword parameters

```
def standard_arg(arg1, arg2):  # by default, arg1 and arg2 are
 ↪  positional-or-keyword
    print("1st argument =", arg1, "| 2nd argument =", arg2)

standard_arg("Hello", "World")             # both by position
standard_arg(arg1="Hello", arg2="World")   # both by keyword
# mixed: positional arguments must come first
standard_arg("Hello", arg2="World")
```

```
1st argument = Hello | 2nd argument = World
1st argument = Hello | 2nd argument = World
1st argument = Hello | 2nd argument = World
```

A positional argument can never follow a keyword argument:

```
standard_arg(arg1="World", "Hello")    # SyntaxError
```

```
SyntaxError: positional argument follows keyword argument (3747978280.py, line
 ↪  1)
  Cell In[86], line 1
    standard_arg(arg1="World", "Hello")    # SyntaxError
                                       ^
SyntaxError: positional argument follows keyword argument
```

Positional-only parameters

```
def pos_only_arg(arg, /):
    print(arg)

pos_only_arg(2)          # fine
```

```
2
```

```
# TypeError: arg is positional-only, so it cannot be passed by keyword
pos_only_arg(arg=2)
```

```
TypeError: pos_only_arg() got some positional-only arguments passed as keyword
 ↪  arguments: 'arg'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[88], line 2
      1 # TypeError: arg is positional-only, so it cannot be passed by keyword
----> 2 pos_only_arg(arg=2)
TypeError: pos_only_arg() got some positional-only arguments passed as keyword
 ↪  arguments: 'arg'
```

Keyword-only parameters

```
def kwd_only_arg(*, arg):
    print(arg)

kwd_only_arg(arg=3)      # fine
```

```
3
```

```
kwd_only_arg(3)          # TypeError: arg is keyword-only, so we must write
 ↪  arg=3
```

```
TypeError: kwd_only_arg() takes 0 positional arguments but 1 was given
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[90], line 1
----> 1 kwd_only_arg(3)          # TypeError: arg is keyword-only, so we must
 ↪  write arg=3
TypeError: kwd_only_arg() takes 0 positional arguments but 1 was given
```

All three kinds combined

```
def combined_example(pos_only_arg, /, standard_arg, *, kwd_only_arg):
    """Demonstrate how arguments are passed to the three kinds of
     ↪  parameters.

    pos_only_arg: positional-only
    standard_arg: positional-or-keyword
    kwd_only_arg: keyword-only
    """
    print(pos_only_arg, standard_arg, kwd_only_arg)

combined_example(1, 2, kwd_only_arg=3)                  # fine
combined_example(4, standard_arg=5, kwd_only_arg=6)     # also fine
```

```
1 2 3
4 5 6
```

```
# TypeError: the first one is positional-only
combined_example(pos_only_arg=1, standard_arg=5, kwd_only_arg=6)
```

```
TypeError: combined_example() got some positional-only arguments passed as
 ↪  keyword arguments: 'pos_only_arg'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[92], line 2
      1 # TypeError: the first one is positional-only
----> 2 combined_example(pos_only_arg=1, standard_arg=5, kwd_only_arg=6)
TypeError: combined_example() got some positional-only arguments passed as
 ↪  keyword arguments: 'pos_only_arg'
```

5 Scope and how arguments are passed

5.1 Local variables and scope

The **scope** of a variable is the part of the program in which it can be used. Variables created inside a function — including its parameters — are **local**: they exist only while the function is running, and they cannot be seen from outside it. Variables created at the top level of a program or notebook are **global**.

```
def compute_area(width, height):
    area = width * height      # area is a local variable
```

```
    return area

print(compute_area(3, 4))
print(area)                    # NameError: area only exists inside the
 ↪  function
```

```
12
```

```
NameError: name 'area' is not defined
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[93], line 6
      2     area = width * height      # area is a local variable
      3     return area
      4
      5 print(compute_area(3, 4))
----> 6 print(area)                    # NameError: area only exists inside the
 ↪  function
NameError: name 'area' is not defined
```

> **Keep functions self-contained**
>
> A function *can* read a global variable, and with the keyword `global` it can even change one. Avoid both: functions are much easier to understand, test and reuse when everything they need comes in through their parameters and everything they produce goes out through `return`.

5.2 What happens to an argument?

When you call a function, Python does **not** copy the arguments. Each parameter simply becomes a new, local name for the object that was passed in — exactly as if we had written the assignment `parameter = argument` (recall the figure on names and objects in Lecture 1). Two different situations can then arise:

1. **Rebinding.** If the function assigns a new value to the parameter, e.g. `a = 20`, only the *local* name is moved to a new object. The caller's variable still refers to the original object and is not affected.
2. **Mutating.** If the function *changes the object itself*, e.g. `names[0] = "Stranger"` for a list, then the one object shared by the caller and the function is modified, and the caller sees the change.

Numbers, strings and tuples are **immutable**: they cannot be changed in place, so a function can never modify them for the caller. Lists and dictionaries are **mutable**, so a function can modify them.

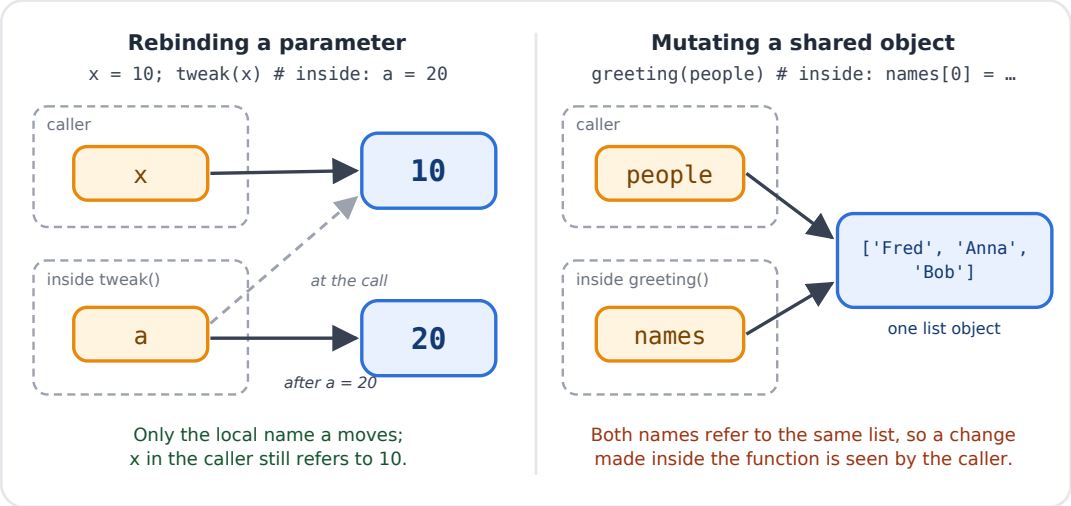


Figure 1: Left: rebinding the parameter `a` inside `tweak()` does not affect the caller's variable `x`. Right: `people` and `names` refer to the same list, so a change made through `names` inside `greeting()` is visible to the caller.

Rebinding a parameter. We define a function that receives one argument and assigns a new value to it inside the function. The variable outside the function does not change.

```
def tweak(a):
    print("inside, before:", a)
    a = 20                          # rebinds the LOCAL name a to a new
 ↪  object
    print("inside, after: ", a)

x = 10
tweak_output = tweak(x)
print("outside:", x)  # x still refers to 10
print(tweak_output)   # tweak() has no return statement, so it returns None
```

```
inside, before: 10
inside, after:  20
outside: 10
None
```

Mutating a list argument. When a function receives a list, it works on the caller's list itself, not on a copy. Recall from Lecture 3 that `list_alias = my_list` creates an alias rather than a copy; passing a list to a function does exactly the same.

```
def greeting(names):
    """Say hello to everyone in the list ... and then change the list!"""
```

```
    print("id(names)  =", id(names))     # the ID of the parameter
    for name in names:
        print("Hello " + name + "!", end="\t")
    print()
    names[0] = "Somebody else"           # change the first element of the
 ↪  list
    names[-1] = "Stranger"               # change the last element of the
 ↪  list

people = ["Fred", "Anna", "Bob"]
print("people (before the call) =", people)
# the same ID as the parameter inside the function
print("id(people) =", id(people))
print(15*"=" + " Function call " + 15*"=")
greeting(people)
print(45*"=")
# the changes are visible after the call
print("people (after the call)  =", people)
```

```
people (before the call) = ['Fred', 'Anna', 'Bob']
id(people) = 129531523368576
=============== Function call ===============
id(names)  = 129531523368576
Hello Fred! Hello Anna! Hello Bob!
=============================================
people (after the call)  = ['Somebody else', 'Anna', 'Stranger']
```

Why doesn't Python copy the arguments? A list can be very large and can occupy a large chunk of memory. Copying it would be expensive in two respects: (i) computing time, because every element must be copied, and (ii) memory, because the copy is as large as the original. Moreover, when we pass a list to a function, we often *want* the function to modify it. A typical example is sorting: the method `my_list.sort()` sorts the list in place, rather than creating a sorted copy, because we usually no longer need the unsorted list. This behaviour resembles *passing by reference* (or *by pointer*) in languages such as C/C++. In Python, we say that a name is **bound** to an object.

If you want to protect the caller's list, pass a copy explicitly:

```
people = ["Fred", "Anna", "Bob"]
greeting(people.copy())        # the function receives (and changes) a copy
 ↪  ...
print(people)                  # ... so the original list is unchanged
```

```
id(names)  = 129530701963328
Hello Fred! Hello Anna! Hello Bob!
```

```
['Fred', 'Anna', 'Bob']
```

> **More resources**
>
> - Ned Batchelder, Facts and myths about Python names and values — an excellent explanation of this topic.
> - Real Python, Pointers in Python: What's the Point?

6 Functions with any number of arguments

Finally, a less frequently used option is to define a function that can be called with an **arbitrary number of arguments**. There are two ways to do this:

1. extra positional arguments, collected in a **tuple** (`*args`);
2. extra keyword arguments, collected in a **dictionary** (`**kwargs`).

6.1 Extra positional arguments: `*args`

Syntax

```
def function_name(normal_arg_1, normal_arg_2, ..., *args):
    <statements>
```

Zero or more normal parameters may come first. The parameter `*args` then *scoops up all the remaining positional arguments* into a tuple called `args` (the name `args` is only a convention). For this reason, `*args` is usually the **last** positional parameter. Any parameters written after `*args` are **keyword-only**. This rule makes perfect sense: since we do not know in advance how many arguments will be passed, any further arguments not given by keyword would be scooped up into the tuple as well. Python cannot read our intentions, so we must make the difference explicit.

```
def print_out_info(name, age, *args):
    print("Name:", name)
    print("Age:", age)
    print("args =", args)            # all the remaining arguments, in a
    ↪ tuple
    print("Gender:", args[0])
    print("Profession:", args[1])
    print("Years of experience:", args[2])

print_out_info("Khiem", 37, "male", "lecturer", "3")
# Note: we cannot write
```

```
#   print_out_info("Khiem", 37, ("male", "lecturer", "3"))
# because the last argument would be understood as ONE element, giving
#   args = (("male", "lecturer", "3"),)
# i.e. a tuple with a single element, which is itself a tuple of three
 ↪  elements.
```

```
Name: Khiem
Age: 37
args = ('male', 'lecturer', '3')
Gender: male
Profession: lecturer
Years of experience: 3
```

A more useful example. We want a function that joins all its arguments, however many there are, with a separator string. First, let us see how the string method `join()` works:

```
my_tuple = "Mech", "Engineering", "Skills", "3"
my_string = "."
print(my_string.join(my_tuple))
# The separator string can also be written directly:
print("--".join(my_tuple))
print("//".join(my_tuple))
print(" ".join(my_tuple))
```

```
Mech.Engineering.Skills.3
Mech--Engineering--Skills--3
Mech//Engineering//Skills//3
Mech Engineering Skills 3
```

Now we define a function that joins all its arguments with a separator, which defaults to `"/"`:

```
def concat(*args, sep="/"):    # anything after *args is keyword-only
    """Join all the arguments with the separator sep."""
    return sep.join(args)

print(concat("earth", "mars", "venus"))   # the default separator "/" is
 ↪  used
```

```
earth/mars/venus
```

```
# Let us try to join "earth", "mars" and "venus" with a full stop "."
```

```
# without passing the separator as a keyword argument, and see what
 ↪ happens:
print(concat("earth", "mars", "venus", "."))       # "." is scooped up into
 ↪ args!
# The result is earth/mars/venus/. because "." is treated as one more item
 ↪ to join.

# To use "." as the separator, we must say so with a keyword:
print(concat("earth", "mars", "venus", sep="."))
```

```
earth/mars/venus/.
earth.mars.venus
```

6.2 Unpacking argument lists with *

The reverse situation occurs when the arguments are already in a list or a tuple, but the function expects separate positional arguments. For instance, the built-in function `range()` expects separate *start* and *stop* arguments. If they are not available separately, write the call with the * operator to **unpack** the arguments from the list or tuple.

```
my_list = list(range(3, 8))
print("my_list  =", my_list)

args = [3, 8]
our_list = list(range(*args))      # the same as range(3, 8)
print("our_list =", our_list)
```

```
my_list  = [3, 4, 5, 6, 7]
our_list = [3, 4, 5, 6, 7]
```

```
planets = ("earth", "mars", "venus", "jupiter", "saturn")
concat(*planets, sep="--")  # the same as concat("earth", "mars", ...,
 ↪ sep="--")
```

```
'earth--mars--venus--jupiter--saturn'
```

6.3 Extra keyword arguments: **kwargs

In the same fashion, a function can receive any number of **keyword arguments**, which are collected into a dictionary with the ** operator.

Syntax

```
def function_name(..., *args, **kwargs):
    <statements>
```

The function can receive normal arguments, then extra positional arguments in a tuple, and finally extra keyword arguments in a dictionary. The last parameter, `**kwargs`, stands for a variable number of keyword arguments: each argument `key=value` in the call becomes one key–value pair of the dictionary `kwargs`. This is clearly different from `*args`, which receives values without names.

```
def print_input_as_dictionary(**kwargs):
    print("All the keys:")
    for key in kwargs.keys():
        print(key, end=",  ")
    print("\n--------------------")
    print("All the values:")
    for value in kwargs.values():
        print(value, end=",  ")
    print("\n--------------------")
    print("All the key-value pairs:")
    for key, value in kwargs.items():
        print("{0}: {1}".format(key, value), end=",  ")
    print("\n--------------------")

# Call the function with four keyword arguments of the form key=value
print_input_as_dictionary(one=1, two=2, name="Coffee", lastname="Milk")
```

```
All the keys:
one,  two,  name,  lastname,
--------------------
All the values:
1,  2,  Coffee,  Milk,
--------------------
All the key-value pairs:
one: 1,  two: 2,  name: Coffee,  lastname: Milk,
--------------------
```

```
def print_arbitrary_inputs(*args, **kwargs):
    # First, print all the inputs collected in the tuple
    print("Inputs collected in a tuple:")
    for element in args:
        print(element, end=",  ")
```

```
    # Then, print all the inputs collected in the dictionary
    print("\n\nInputs collected in a dictionary:")
    for key, value in kwargs.items():
        print("{0} = {1}".format(key, value), end=",  ")

# Call the function with 3 positional arguments and 3 keyword arguments
print_arbitrary_inputs("I", "am", "weird", eleven=11, twelve=12, ten=10)
```

```
Inputs collected in a tuple:
I,  am,  weird,

Inputs collected in a dictionary:
eleven = 11,  twelve = 12,  ten = 10,
```

6.4 Unpacking dictionaries with `**`

Conversely, a dictionary can deliver keyword arguments to a function with the `**` operator:

```
def describe_beam(length, width, height):
    print(f"Beam: {length} m long, {width} m wide, {height} m high")

dimensions = {"length": 2.0, "width": 0.1, "height": 0.2}
# the same as describe_beam(length=2.0, width=0.1, height=0.2)
describe_beam(**dimensions)
```

```
Beam: 2.0 m long, 0.1 m wide, 0.2 m high
```

Summary

- Functions let us reuse code, break big problems into small tasks, and build on other people's work. Define them with `def`, and document them with a docstring.
- `return` hands a value back to the caller; a function without `return` returns `None`. Return several values as a tuple.
- Arguments can be passed by position or by keyword; parameters can have default values (never a mutable one). `/` and `*` mark positional-only and keyword-only parameters.
- Variables created inside a function are local. Arguments are not copied: rebinding a parameter does not affect the caller, but mutating a mutable argument (such as a list) does.
- `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dictionary; `*` and `**` in a call unpack a sequence or a dictionary into separate arguments.

Exercises

Exercise 1: A guessing game

Write a **guessing game**. Your program should:

- let the computer pick a random integer between 1 and 32 (inclusive) as the answer — see Exercise 5 of Lecture 2 for the `random` module;
- define a function `check_guess(correct_answer)` that
    1. has one integer parameter, `correct_answer`;
    2. reads an integer from the keyboard;
    3. compares the number entered with the correct answer;
    4. prints the appropriate message ("correct", "too high" or "too low");
    5. returns `True` if the guess was correct and `False` otherwise;
- give the user 5 chances to guess the answer;
- end the game when the user guesses correctly or has used up all 5 chances, printing "You win!" or "You lose!" as appropriate.

> Hint
>
> Print the random number before the game starts while you are testing your code. It is tempting to copy and paste the call to `check_guess()` five times — but the program would not scale if the task asked for 10 or 100 chances. Use a loop with `break` instead.

Exercise 2: Projectile motion as a function

Revisit Exercise 6 of Lecture 1. Write a function `projectile(v0, theta_deg, g=9.81)` that returns a tuple `(T, h_max, d)` containing the time of flight $T = 2v_0 \sin\theta / g$, the maximum height $h_{\max} = (v_0 \sin\theta)^2 / (2g)$, and the range $d = v_0 \cos\theta \; T$, for a launch speed `v0` (in m/s) and a launch angle `theta_deg` (in degrees).

1. Call the function for $v_0 = 7.5$ m/s and $\theta = 25^\circ$, unpack the result, and print the three values with three decimal places.
2. Use a loop to find the launch angle between 1° and 89° (in steps of 1°) that gives the longest range. Is the result what you expected?
3. How does the range change on the Moon, where $g = 1.62\,\text{m/s}^2$? Use the keyword argument `g`.

Exercise 3: Resistors in series and in parallel

The total resistance of resistors connected in series is $R = R_1 + R_2 + \cdots + R_n$, and in parallel it satisfies $1/R = 1/R_1 + 1/R_2 + \cdots + 1/R_n$. Write a function

```
def total_resistance(*resistors, connection="series"):
    ...
```

that accepts any number of resistances and returns the total resistance for `connection="series"` or `connection="parallel"`. Test it with $100\,\Omega$, $220\,\Omega$ and $330\,\Omega$. What happens if you call `total_resistance(100, 220, "parallel")`, and why?

Exercise 4: Functions with and without side effects

1. Write a function `describe(numbers)` that returns the mean, the minimum and the maximum of a list of numbers as a tuple.
2. Write a function `normalise(numbers)` that returns a **new** list in which every value x is replaced by $(x - x_{\min})/(x_{\max} - x_{\min})$, so that all values lie between 0 and 1. Check that the original list is unchanged after the call.
3. Write a function `normalise_in_place(numbers)` that modifies the list it receives and returns nothing. Check that the caller's list *has* changed.
4. In your own words, explain the difference between the two versions, and give one situation in which each of them is the better choice.

Further reading

- The official Python tutorial: Defining Functions and More on Defining Functions.
- PEP 257 — Docstring Conventions.